

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – ARCHITECTURE / MODELING (DEVSECOPS METHODOLOGY)

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Develop Complete DevSecOps Architecture, Methodology Documentation and GitHub Submission.		
Student Name:	K.Yalla Reddy	Reg. No.:	192465047
Section / Batch:	2024	Date:	26-08-2026

Instructions to Students

- Develop a complete and original DevSecOps architecture showing the end-to-end secure software delivery workflow.
- Include development, source control, CI/CD, automated security testing, artifact management, deployment, monitoring and feedback stages.
- Document the methodology clearly, including security controls, tools, workflow relationships and key design decisions.
- Use clear labels, consistent symbols, proper alignment and professionally formatted architecture diagrams. • Submit the editable architecture source files, methodology documentation and an accessible GitHub repository containing all required artifacts.

ARCHITECTURE / MODELING DevSecOps

Methodology Q1 Q1 – 100

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Complete DevSecOps Architecture Design	20%	Develops a comprehensive and technically accurate DevSecOps architecture integrating development, security, CI/CD, infrastructure and operations components	Develops a clear DevSecOps architecture containing most required components with only minor technical or integration gaps	Develops a basic DevSecOps architecture with limited integration or several missing components	Provides an incomplete or technically inaccurate DevSecOps architecture with weak component integration
Security Integration Across DevSecOps Lifecycle	30%	Effectively integrates security controls and automated security testing across planning, coding, build, test, deployment and operations stages	Integrates appropriate security controls across most DevSecOps stages with only minor omissions	Shows basic security integration but several lifecycle stages or security controls are missing	Shows limited or incorrect security integration across the DevSecOps lifecycle

Methodology and Workflow Documentation	20%	Clearly documents the complete DevSecOps methodology, workflow, tools, responsibilities and security checkpoints with strong technical justification	Documents the DevSecOps methodology and workflow clearly with minor gaps in explanation or justification	Provides basic methodology documentation with limited workflow details or technical explanation	Provides incomplete or unclear methodology documentation with weak technical reasoning
GitHub Repository and Submission Quality	20%	Submits a complete, well organized GitHub repository containing architecture artifacts, documentation and required files with clear structure and version history	Submits a properly organized GitHub repository containing most required artifacts with minor structural or documentation issues	Submits a partially organized repository with missing artifacts, limited documentation or inconsistent file structure	GitHub submission is incomplete, poorly organized or missing essential architecture and methodology artifacts
Technical Clarity and Professional Presentation	10%	Presents the DevSecOps architecture, methodology and repository professionally with clear relationships, consistent terminology and strong technical clarity	Presents the work clearly and professionally with only minor clarity or formatting issues	Presents the work with basic clarity but noticeable inconsistencies or limited technical explanation	Presents the work poorly with unclear relationships, inconsistent terminology or insufficient technical explanation

Rubrics:

Criteria	Weightage	Q5 (10)	Total Marks (50)
Complete DevSecOps Architecture Design	25%		
Security Integration Across DevSecOps Lifecycle	30%		
Methodology and Workflow Documentation	20%		
GitHub Repository and Submission Quality	15%		
Technical Clarity and Professional Presentation	10%		
Total per Question	100 Marks		

100

1. Abstract

The Shift-Left Secure SDLC Platform integrates application security from planning and coding through production. Automated security tools scan code, dependencies, secrets, infrastructure and containers. A centralized Policy Gatekeeper evaluates these results against policy-as-code rules and decides whether the pipeline can continue. The platform provides early developer feedback, blocks high-risk releases and creates auditable security evidence.

2. Problem Statement

Security testing is often performed late in the SDLC, allowing vulnerabilities to survive until testing or production. This increases remediation cost and release risk. The proposed platform moves security activities closer to development and uses automated policy gates to prevent insecure code and artifacts from progressing.

3. Objectives

- Integrate security into every SDLC stage.
- Automate SAST, SCA, secret, IaC, container and DAST checks.
- Enforce security policies using a central Policy Gatekeeper.
- Give developers fast and actionable feedback.
- Block releases that violate mandatory security requirements.
- Maintain versioned policies, scan results and audit evidence.

4. Complete DevSecOps Architecture

Shift-Left Secure SDLC - DevSecOps Architecture

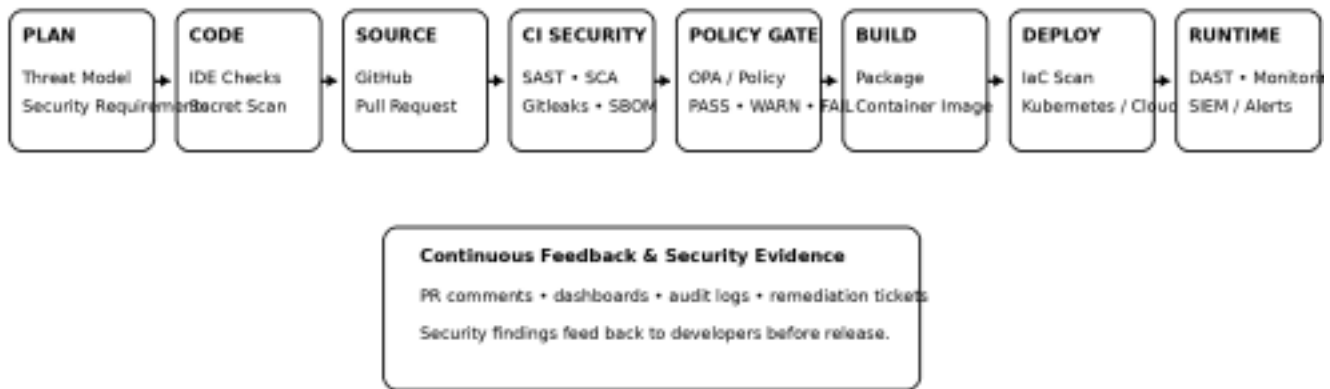


Figure 1: End-to-end Shift-Left Secure SDLC architecture.

5. Architecture Components

- Plan: security requirements, threat modeling and risk classification.
- Code: secure coding practices, IDE checks and secret prevention.
- Source Control: Git repository, pull requests, branch protection and code review.
- CI Security: SAST, SCA, secret scanning, SBOM generation and unit tests.
- Policy Gatekeeper: evaluates scan results and enforces mandatory policy rules.
- Artifact Management: stores approved packages and container images.
- Deployment: IaC scanning, least privilege and Kubernetes/cloud admission controls.
- Runtime: DAST, monitoring, logging, alerting and vulnerability management.
- Feedback: security dashboards, PR comments, tickets and audit records.

6. Policy-Based Gatekeeping

The Policy Gatekeeper is the main enforcement point. It converts security requirements into measurable rules and returns PASS, WARN or FAIL. A FAIL blocks promotion of the build or deployment.

Policy Gatekeeping - Release Decision Flow



Example mandatory rules:

- No Critical CVEs
- No exposed secrets
- Required scans must run
- High-risk IaC issues blocked

Figure 2: Policy Gatekeeping decision flow.

7. Example Security Policies

Policy Condition Decision Action Critical CVEs Critical vulnerabilities > 0 FAIL Stop pipeline

Credential detected FAIL Block and rotate SAST Critical finding unresolved FAIL Fix before merge

SCA Unapproved deployment

dependency/license

IaC Sensitive resource publicly exposed

WARN/FAIL Review or replace FAIL Block

Container Critical image CVE FAIL Rebuild image

FAIL Fix before release

DAST Confirmed High/Critical issue

Exception Approved and time-bound WARN/PASS Record and monitor

8. CI/CD Secure Workflow

1. Developer creates a feature branch and writes code.

- 2. Local checks scan for secrets and common security issues.
- 3. Pull request triggers CI security checks.
- 4. SAST, SCA, secret scanning and SBOM generation execute.
- 5. Policy Gatekeeper evaluates all mandatory rules.
- 6. Failed gates stop the pipeline and return findings to the developer.
- 7. Passed builds are packaged, scanned and stored as controlled artifacts.
- 8. Deployment validates IaC and admission policies.
- 9. DAST and runtime security checks continue after deployment.
- 10. Monitoring feeds security events and lessons back into development.

9. Sample Security Dashboard Output

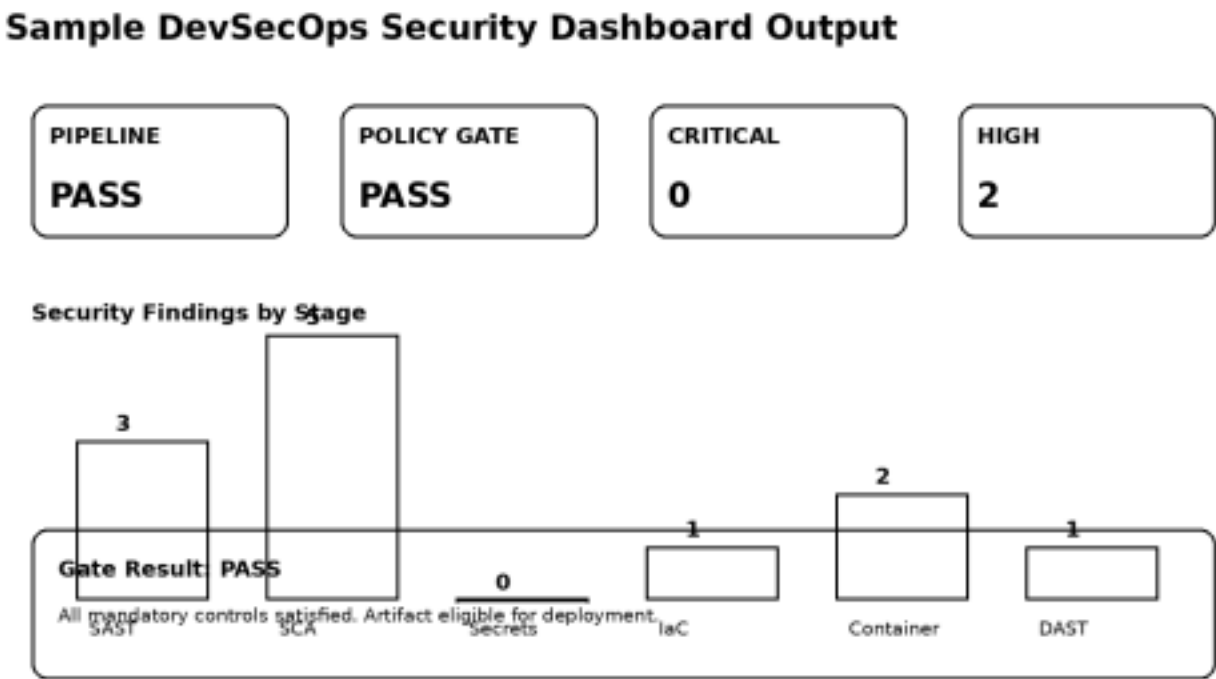


Figure 3: Example output showing pipeline status, policy result and security findings.

10. Tools and Technologies

Layer Technology Purpose

Source Control GitHub Version control and pull requests

CI/CD GitHub Actions Automated build and security pipeline

SAST CodeQL / Semgrep Source-code security analysis SCA OWASP Dependency-Check / Trivy

Dependency vulnerability scanning Secrets Gitleaks Credential detection IaC Checkov Infrastructure security scanning Container Trivy Image vulnerability scanning DAST OWASP ZAP Dynamic application testing Policy

OPA / Conftest Policy-as-code gatekeeping Monitoring Prometheus / Grafana Operational visibility Logs ELK /

SIEM Security event analysis

11. GitHub Repository Structure

GitHub Repository - Recommended Structure

```
shift-left-secure-sdlc/  
├─ README.md  
├─ architecture/  
│   └─ devsecops-architecture.drawio  
│   └─ architecture.png  
├─ policies/  
│   └─ security-policy.rego  
│   └─ policy-exceptions.md  
├─ .github/workflows/  
│   └─ secure-pipeline.yml  
├─ security/  
│   └─ threat-model.md  
│   └─ security-checklist.md  
├─ docs/  
│   └─ methodology.md  
│   └─ gatekeeping.md  
└─ reports/  
    └─ sample-security-report.md
```

Figure 4: Recommended repository structure for the project.

12. Roles and Responsibilities

- Developer – writes secure code and fixes security findings.
- Reviewer – validates pull requests and secure design.
- DevSecOps Engineer – maintains CI/CD automation and security tools.
- Security Team – defines policies, thresholds and exception rules.

- Operations Team – manages secure deployment, monitoring and response.
- Management/Product Owner – approves justified and time-bound risk exceptions.

13. Advantages

- Early detection reduces remediation cost.
- Automated gates prevent vulnerable releases.
- Policy-as-code gives consistent security decisions.
- Security becomes a shared development responsibility.
- Audit evidence and version history improve traceability.
- Continuous feedback supports ongoing improvement.

14. Conclusion

The Shift-Left Secure SDLC Platform provides a complete DevSecOps architecture by embedding security into the full software delivery lifecycle. The Policy Gatekeeper acts as the central control mechanism that evaluates automated security results and enforces release policies. This approach improves security, developer feedback, compliance and delivery confidence.

15. References

- OWASP – Secure Software Development and Application Security resources.
- NIST SP 800-218 – Secure Software Development Framework (SSDF).
- Open Policy Agent (OPA) – Policy as Code.
- GitHub Actions – CI/CD automation documentation.
- OWASP ZAP – Dynamic Application Security Testing.